Lesson Plan
Course Code: CS2024
Name of the Programme: B.Tech(H)
Name of the Course: Object Oriented Programming with Java
Semester: IV

Course credit	**No. of hours per week (L + T + P) - 2 - 1 - 2**	**Total no. of Teaching hours - 75**

Course Objectives:

a) Introduce the fundamentals of Java and core object-oriented programming concepts such as classes, inheritance, and polymorphism.

b) To promote modular programming through the use of packages, interfaces, and UML design principles.

c) To equip students with the skills to handle exceptions and implement multithreaded Java programs.

d) To familiarize students with strings, collections, file handling, and object serialization in Java.

Course outline (Syllabus of the course) - Template - 1

Syllabus:

Module - 1	**No. of hours**
	6
Introduction to Object-Oriented Programming (OOP); Comparison of Procedural, Object-Oriented, and Scripting Languages; History and Architecture of Java; Features of Java; Java Keywords, Data Types, Variables, Operators, Control Statements, and Arrays.	

Module - 2	**No. of hours**
	6
Classes and Objects, Methods, Constructors, Overloading Constructors, this keyword, Garbage Collection, finalize() method. Inheritance, Super keyword, Polymorphism – Method Overloading, Method Overriding, Access Modifiers, Encapsulation and Getter/Setter methods, Data Abstraction, Abstract Classes, final – keyword, method and class.	

Module - 3	**No. of hours**
	6
Packages, Creating a Package, Finding Packages and Classpath, Importing Packages, Interfaces, Creating and Implementing Interfaces, Variables in Interfaces, Extending Interfaces, Introduction to UML Class Diagrams, Creating Class Diagrams to Represent Classes, Interfaces, Attributes, Methods, and Relationships.	

Module - 4	No. of hours
	6
Exception Handling Fundamentals – Exception types, Using try and catch, Multiple catch clauses, Nested try statements, Java's built-in exceptions, Creating custom exceptions. Multithreaded Programming – Java thread model, Creating a thread, Creating multiple threads, Thread priorities, Synchronization, Inter-thread communication.	

Module - 5	No. of hours
	6
String Constructors, String Operations, Wrapper Classes and Autoboxing/Unboxing, The Collection Framework, Collections, List – ArrayList, LinkedList, Set – HashSet, Queue – PriorityQueue, Map – HashMap, TreeMap, Files and Streams – Byte Streams and File I/O, Character Stream – FileReader and FileWriter, Serialization.	

Course Outcomes: After completing the course, the students will be able to:	
CO1	Develop Java applications using fundamentals and object-oriented programming concepts such as classes, inheritance, and polymorphism.
CO2	Implement modular design using packages, interfaces, and UML class diagrams.
CO3	Apply exception handling and multithreading mechanisms to design robust and concurrent Java programs.
CO4	Apply string operations, collection framework, file I/O, and serialization in java applications.

Text Books	
1	H. Schildt, *Java: The Complete Reference*, 12th ed. New Delhi, India: Tata McGraw Hill, 2022. ISBN: 978-1260463415.
2	P. Deitel and H. Deitel, *Java: How to Program (Early Objects)*, 11th ed. Boston, MA: Pearson Prentice Hall, 2017. ISBN: 978-0134743356.

Reference Books

1	E. Balagurusamy, *Programming with Java*, 7th ed. New Delhi, India: Tata McGraw Hill, 2019. ISBN: 978-935316234.
2	Y. D. Liang, *Introduction to Java Programming*, 12th ed. Boston, MA: Pearson Education, 2018. ISBN: 978-0134670942.

Tutorials [15 Hours]	
1	Comparison of Procedural, Object-Oriented, and Scripting Programming Paradigms
2	Java Architecture and Execution Model
3	Access Modifiers and Package-Level Visibility
4	Representation of Object-Oriented Relationships using UML Class Diagrams
5	Java Thread Lifecycle and State Transitions
6	Wrapper Classes and the Autoboxing/Unboxing Mechanism
7	Serialization and Deserialization

Lab Programs [30 Hours]	
1	Develop basic Java programs using classes, objects, and methods.
2	Demonstrate constructor overloading and use of this keyword.
3	Design and implement different types of inheritance through classes in Java.
4	Demonstrate method overloading and method overriding in a Java application.
5	Implement an abstract class and define its behavior in a concrete subclass.
6	Develop a Java program using interfaces and demonstrate multiple and hybrid inheritance through interface implementation.
7	Create and use user-defined packages across multiple Java files.
8	Handle built-in exceptions using try-catch-finally and multiple catch blocks.
9	Implement user-defined exceptions with appropriate use of throw and throws.
10	Create threads using both Thread class and Runnable interface and execute them concurrently.
11	Demonstrate inter-thread communication using wait(), notify(), notifyAll() and synchronization blocks.

12	Use built-in methods of the String class for string manipulation operations.
13	Implement collections like List, Set, Queue, Map in java.
14	Develop java programs to access and perform various operations in file contents.
15	Implement a real-world use case using various object-oriented concepts in Java.

Lesson plan (template – 2)

Name of the Programme:	B.Tech(H)	
Title of the Course:	Object Oriented Programming with Java	
Course code:	CS2024	
Semester:	IV	
Names of the Course Instructor(s):	**Dr.Deepika S**, Dr.Srinivas Mishra, Prof. Lokesh J K, Dr. Roopesh R, Prof. Rahul Panja	
Course credit : 4	**No. of hours per week (L + T + P) :** 2 - 1 - 2	**Total no. of Teaching hours**
		75

Session	Module No.	Topic	Pedagogy / Activities	Pre-reading / Reference
1	**1**	Introduction to OOP and need	Lecture with real-life examples	Schildt Ch.1
2		Procedural vs OOP vs Scripting languages	Comparative discussion	Balagurusamy Ch.1

3		Programming paradigms	Tutorial: problem discussion	
4		Basic Java program structure	Lab: simple Java programs	Lab Manual
5		History and evolution of Java	Timeline-based explanation	Schildt Ch.1
6		Features of Java	Lecture + examples	Schildt Ch.1
7		Java architecture	Tutorial: JVM diagram analysis	Schildt Ch.2
8		Variables and data types	Lab: coding practice	Lab Manual
9		Operators and expressions	Board problems	Deitel Ch.4
10		Control statements	Lecture with flowcharts	Deitel Ch.4
11		Control flow problems	Tutorial: problem solving	
12		Arrays	Lab: array programs	
13	**2**	Classes and objects	UML-based explanation	Schildt Ch.6
14		Methods	Live coding	Deitel Ch.8
15		Access modifiers	Tutorial: scenario discussion	Schildt Ch.6

16		Classes and methods	Lab: implementation	
17		Constructors	Lecture with examples	Schildt Ch.6
18		Constructor overloading and this	Code tracing	Schildt Ch.6
19		Constructor problems	Lab Coding : tracing exercises	
20		Constructor programs	Lab: hands-on coding	
21		Inheritance	UML hierarchy explanation	Deitel Ch.9
22		super keyword	Live coding	Deitel Ch.9
23		Inheritance problems	Lab:Program Solving	
24		Inheritance programs	Lab: coding	
25		Polymorphism	Runtime vs compile-time demo	Deitel Ch.10
26		Method overriding	Code demonstration	Deitel Ch.10
27		Polymorphism problems	Lab:Coding	
28		Polymorphism programs	Lab: implementation	

29		Encapsulation and abstraction	Case study	Balagurusamy Ch.5
30		Abstract classes and final keyword	Lecture + examples	Schildt Ch.6
31		Abstraction concepts	Lan:Lecture + concept explanation	
32		Abstract class programs	Lab: coding	
33	**3**	Packages	Lecture with folder demo	Schildt Ch.9
34		Creating and importing packages	Live coding	Schildt Ch.9
35		Package-level visibility	Lab: Package implementation	
36		Package programs	Lab: multi-file coding	
37		Interfaces	Real-world example	Deitel Ch.10
38		Implementing interfaces	Live coding	Deitel Ch.10
39		Interface vs abstract class	Lab Discussion	
40		Interface programs	Lab: implementation	
41		UML class diagrams	Diagram explanation	Deitel UML

42		UML relationships	Lecture with examples	Deitel UML
43		UML practice	Lab:Coding	
44		UML-based Java program	Lab: design to code	
45	**4**	Exception handling fundamentals	Lecture with hierarchy	Schildt Ch.10
46		try-catch and multiple catch	Error demo	Schildt Ch.10
47		Exception handling problems	Lab:Coding	
48		Built-in exception programs	Lab: coding	
49		User-defined exceptions	Lecture + coding	Deitel Ch.11
50		Java thread model & Life cycle	Lifecycle explanation	Schildt Ch.11
51		User-defined exception programs	Lab: implementation	
52		Thread creation techniques	Lecture	Deitel Ch.23
53		Synchronization	Case study	Schildt Ch.11
54		Multithreading issues	Lab:Discussion of the concept	
55		Multithreading programs	Lab: coding	

56	**5**	String constructors & String operations	Lecture + immutability & memory-level explanation	Schildt Ch.16
57		Wrapper classes & Autoboxing/Unboxing	Lecture + code snippets	Deitel Ch.12
58		Collection Framework overview: List, Set, Queue, Map	Lab:Coding	Deitel Ch.18
59		List (ArrayList, LinkedList), Set (HashSet), Queue (PriorityQueue), Map (HashMap, TreeMap)	Lecture + real-world examples	Deitel Ch.18
60		Files & Streams (Byte & Character), Serialization	Lab: File I/O + object serialization	Schildt Ch.13

Assessment and Evaluation

Continuous Internal Evaluation (CIE): 70%		
Components	**Details of Assessment**	**Weightage**
CIE-1	● Pen and Paper Test ● Quiz/ MOOC.	15 marks out of 70 05 marks out of 70
CIE-2	● Theory Test: Mid-semester written examination.	25 marks out of 70
CIE-3	● Lab Record Write-up & Viva: Evaluation of weekly lab performance.	10 marks out of 70

	• Mini-Project: Report Submission • Mini Project: Final code, and Presentation. • Mini Project: Viva	03 marks out of 70 07 marks out of 70 05 out of 70
Total		**70 Marks**

Semester End Exams (SEE): 30%	
Mode of Exam	Theory(Pen & Paper)
Weightage of exam	30%

Course Outcomes- Programme Outcomes Matrix

Sl. No	Course Outcomes	PO 1	PO 2	PO 3	PO 4	PO 5	PO 6	PO 7	PO 8	PO 9	PO 10	PO 11	PO 12
1	**CO1**	**3**	**2**	**1**	**2**	–	–	–	**2**	–	–	–	–
2	**CO2**	**3**	**2**	**2**	**3**	**1**	**2**	**2**	**2**	–	–	–	–
3	**CO3**	**3**	**3**	**2**	**3**	–	**2**	**1**	**2**	–	–	–	–
4	**CO4**	**3**	**2**	**1**	**3**	–	**1**	**2**	**3**	–	–	–	–

Rubrics for evaluation of CIE-1

Programme Outcome	As per PO Mapping
Total Marks	**20**

CIE Components	CO–PO	Excellent	Very Good	Good	Average	Needs Improvement
Pen & Paper Test **(10 Marks)**	CO1–PO1, PO2	Demonstrates complete understanding of Java fundamentals and OOP concepts with accurate answers (10-9 Marks)	Shows strong understanding with minor conceptual errors (8-6 Marks)	Understands basic concepts but lacks depth (5-3 Marks)	Partial understanding with noticeable errors (2-1 Marks)	Very limited understanding of concepts (1-0 Marks)
Quiz / MOOC **(5 Marks)**	CO1–PO1, PO4	Answers almost all questions correctly with clear conceptual clarity (5 Marks)	Correctly answers most questions (4 Marks)	Answers moderate number of questions correctly (3 Marks)	Limited correct responses (2 Marks)	Unable to answer most questions (1 Marks)

Rubrics for evaluation of CIE-2

Programe Outcome	As per PO Mapping
Total Marks	**25**

CIE Components	**CO–PO**	**Excellent (22-25)**	**Very Good (16-21)**	**Good (10-15)**	**Average (4-9)**	**Needs Improvement (1-3)**
Mid-Sem Theory Exam (25 Marks)	CO1, CO2 – PO1, PO2, PO4	Demonstrates strong conceptual understanding and applies OOP principles correctly	Minor conceptual gaps but good application ability	Basic understanding with limited application	Partial knowledge with weak application	Poor understanding and incorrect application

Rubrics for evaluation of CIE-3

Programe Outcome	As per PO Mapping
Total Marks	**25**

CIE Components	CO–PO	Excellent (5)	Very Good (4)	Good (3)	Average (2)	Needs Improvement (1)
Lab Record (5 Marks)	CO3, CO4 – PO1, PO4	Executes lab programs accurately and explains concepts confidently	Executes programs with minor guidance	Completes programs with partial understanding	Completes tasks with significant errors	Unable to complete programs
Lab Viva (5 Marks)	CO3, CO4 – PO1, PO2, PO4	Demonstrates complete understanding of lab programs, clearly explains logic, execution flow, and output with confidence	Explains program logic and output correctly with minor gaps	Explains basic logic and execution with limited clarity	Shows partial understanding and struggles to explain program	Unable to explain program logic or lab concepts
Project Report (5 marks)	CO4–PO7, PO8	Well-structured report with clear design and documentation	Good structure with minor issues	Adequate documentation	Poor organization	Incomplete or unclear report

Project Code & Presentation (5 Marks)	CO4–PO1, PO6	Fully functional code with confident presentation	Mostly correct implementation	Partially functional code	Limited functionality	Non-functional implementation
Project Viva (5 Marks)	CO3, CO4 – PO2	Explains design decisions and implementation clearly	Explains most concepts correctly	Basic explanation	Limited clarity	Unable to justify work